

NUTREXIA

D2C E-Commerce Web App — Implementation Plan & Technical Specification

Prepared for: Client Review

Project approach: Single responsive Web App / PWA with a shared Node.js backend and PostgreSQL database.

1. Executive Summary

The Nutrexia platform will be developed as a modern Direct-to-Consumer (D2C) e-commerce Web App. Instead of maintaining separate web and mobile frontends at launch, the platform will use one responsive frontend that works across desktop, tablet and mobile devices and can be installed as a Progressive Web App (PWA).

This approach reduces development and maintenance overhead while still providing an app-like mobile experience. A native React Native application can be introduced later if business requirements justify native-only capabilities.

2. Reference Design & Initial Product Scope

The supplied Nutrexia baseline HTML is treated as the visual/content starting point. It currently presents a long-form D2C landing experience with a launch/founding-price section, shop, formulation/nutrition information, story, climate impact, testimonials, FAQ, newsletter/waitlist and a cart drawer.

- Initial catalogue: Trial Pack — 5 × 30g sachets; 1kg Pouch — approximately 33 servings; Monthly Subscription — 1kg auto-delivered.
- The baseline currently uses client-side/demo cart behaviour; production commerce will move cart, pricing, inventory, checkout and orders to the backend.
- The baseline includes founding-price messaging, countdown/stock presentation and product comparison content; these should become configurable rather than hardcoded.

3. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	Next.js + React + TypeScript	Responsive storefront, SEO, routing, SSR/SSG and PWA
Styling/UI	Modern responsive component system	Reusable Nutrexia design system and mobile-first UI
Backend	Node.js + TypeScript	Commerce APIs, authentication, orders, payments, subscriptions
API framework	NestJS (recommended)	Structured, scalable backend architecture
Database	PostgreSQL	Products, customers, carts, orders, payments, subscriptions, inventory
ORM	Prisma	Type-safe database access and migrations
Caching/queues	Redis (recommended)	Caching, rate limits and background jobs where required
Payments	Razorpay	UPI/cards/net banking and payment processing
Shipping	Shiprocket or equivalent	Pincode serviceability, labels, shipment and tracking
Storage/CDN	S3-compatible object storage + CDN	Product images, documents and lab reports
Analytics	GA4 + Meta tracking	Conversion and campaign measurement

4. High-Level Architecture

One frontend, multiple device experiences:

Customer Device	Application
Desktop / Laptop	Next.js Web App
Tablet	Same responsive Web App
Android / iPhone browser	Same responsive Web App
Installed mobile experience	Same Web App installed as PWA

- The frontend communicates with one Node.js API.
- PostgreSQL is the source of truth for products, prices, customers, orders, inventory and subscriptions.
- Payment and shipping providers are integrated through backend services so provider changes do not require rebuilding the storefront.

5. Customer-Facing Pages

Page	Purpose
/	Conversion-focused homepage based on the supplied Nutrexia baseline
/shop	Product listing and catalogue
/product/[slug]	Full product detail and purchase experience
/collections	Future-ready category/collection structure
/our-story	Brand and founder story
/nutrition	Nutrition, ingredients and formulation information
/lab-reports	Lab reports, safety documentation and certifications
/climate-impact	Climate and sustainability content
/reviews	Customer reviews and social proof
/faq	Frequently asked questions
/contact	Customer support/contact
/blog	Future content/SEO marketing
/cart	Cart and checkout entry
/checkout	Address, delivery, discount and payment
/account	Customer profile, orders, addresses, wishlist and subscriptions
/track-order	Order tracking
Policy pages	Shipping, returns/refunds, cancellation, privacy, terms and disclaimer

6. Core E-Commerce Functionality

- Product catalogue with products, variants, images, pricing, sale pricing, nutrition, ingredients, stock and SEO metadata.
- Guest cart plus authenticated cart; merge guest cart into customer account after login.
- Wishlist / save for later.
- Coupon and discount engine.
- Free-shipping threshold and shipping calculation.
- Secure checkout with backend-calculated totals.

- Razorpay payment integration with server-side verification and webhooks.
- Order confirmation, invoice/receipt support and customer order history.
- Shipping integration with pincode serviceability and shipment tracking.
- Returns, cancellations and refund workflows.

7. Product Detail Experience

- Product image gallery and zoom.
- Product name, rating, pricing, MRP/discount and serving economics.
- Variant/flavour/pack selection where applicable.
- Quantity and Add to Cart / Buy Now.
- Subscription purchase option.
- Benefits, ingredients, nutrition facts and usage.
- Allergen/safety information.
- Lab reports and certifications.
- Reviews and verified-purchase indicators.
- FAQ and related/recommended products.

8. Subscription System

The subscription model will be implemented as a real commerce feature rather than as a static product card. Customers should be able to manage their subscription from their account.

- Create, pause, resume and cancel subscription.
- Change quantity, delivery frequency, address and eligible product/variant.
- Display next billing/renewal and shipment information.
- Store subscription status and payment references.
- Support founding-price rules where commercially applicable.

9. Customer Account

- Registration/login and secure session management.
- Profile and contact information.
- Multiple saved delivery addresses.
- Order list and order detail.
- Shipment tracking.
- Wishlist.
- Subscription management.
- Review submission.
- Notification/preferences settings.

10. Admin Panel

- Dashboard: sales, orders, customers, products, subscriptions and inventory.
- Product management: products, variants, pricing, images, nutrition, ingredients, SEO and publication status.

- Order management: payment, fulfilment, shipment, cancellation, return and refund status.
- Inventory management and stock movements.
- Customer management.
- Coupon/discount management.
- Reviews and testimonial moderation.
- FAQ, blog and content management.
- Homepage/marketing content management so content is not hardcoded into the frontend.
- Admin roles and permissions.

11. Inventory & Batch Traceability

Because Nutrexia is a packaged nutrition product, the system should be designed to support batch-level traceability. This is recommended even if only a small number of products are launched.

- Batch number.
- Manufacturing date.
- Expiry date.
- Quantity produced and available.
- Lab report / COA references.
- Production status.
- Inventory linkage.
- Optional QR code on packaging to expose batch information and documentation.

12. Suggested Database Model

Area	Core Tables / Entities
Users	users, user_profiles, addresses
Catalogue	products, product_variants, product_images, categories
Inventory	inventory, inventory_movements, batches
Commerce	carts, cart_items, orders, order_items, order_addresses
Payments	payments, refunds
Subscriptions	subscriptions, subscription_items
Marketing	coupons, coupon_usage, newsletter_subscribers
Engagement	wishlists, wishlist_items, reviews, notifications
Content	blog_posts, faqs, lab_reports, certifications
Security	audit_logs

13. PWA / Mobile Web Strategy

- Responsive mobile-first interface across common phone, tablet and desktop breakpoints.
- Web App Manifest with Nutrexia name, icon, theme and standalone display.
- Service worker and controlled caching for app shell and suitable static assets.
- Installable experience where browser/OS support permits.
- App-style navigation and mobile-friendly checkout.

- Push notifications can be added subject to browser/device support and user permission.
- The PWA does not require maintaining a separate React Native frontend at launch.

14. SEO & Performance

- Server-rendered/static-rendered public pages using Next.js.
- Unique metadata, canonical URLs and Open Graph data.
- Product, Organization, Breadcrumb, FAQ and review structured data where applicable.
- XML sitemap and robots configuration.
- Optimised product images and lazy loading.
- Core Web Vitals/performance monitoring.
- Indexable product and content pages.

15. Security

- HTTPS throughout production.
- Secure authentication/session or token strategy.
- Password hashing if passwords are supported.
- Role-based admin authorization.
- API rate limiting and request validation.
- Secure cookies and appropriate security headers.
- Payment signature/webhook verification.
- Server-side price, discount, stock and order validation.
- Environment variables for secrets; no credentials in frontend code.
- Audit logs for important administrative actions.
- Database backups and recovery procedures.

16. Analytics & Marketing Automation

Recommended customer events:

- page_view
- product_view
- add_to_cart
- remove_from_cart
- begin_checkout
- payment_started
- purchase
- subscription_started
- subscription_cancelled

These events can support GA4, Meta campaign measurement and future abandoned-cart, email, WhatsApp and push automation.

17. Project Structure

nutrexia/

- apps/web/ # Next.js storefront + PWA
- apps/api/ # Node.js backend
- apps/admin/ # Admin panel (can share Next.js infrastructure)
- packages/ui/ # Shared UI components
- packages/types/ # Shared TypeScript types
- packages/validation/ # Shared validation schemas
- packages/utils/ # Shared utilities
- prisma/ # PostgreSQL schema + migrations

18. Implementation Phases

Phase	Scope	Indicative Duration*
0	Discovery, requirements, product data, policies, integrations and final architecture	1–2 days
1	Project foundation, repositories, environments, database and API foundation	2–4 days
2	Design system and conversion of baseline HTML into reusable React components	3–5 days
3	Storefront: home, shop, product pages, search, cart and wishlist	5–8 days
4	Authentication and customer account	2–3 days
5	Checkout, Razorpay and shipping integration	4–6 days
6	Orders, tracking, cancellation/refund workflows	3–5 days
7	Admin panel and inventory	5–8 days
8	Subscription engine and customer subscription management	3–5 days
9	Analytics, marketing tools, CMS/content controls	3–5 days
10	PWA installation, mobile UX optimisation and notification foundation	3–5 days
11	QA, security testing, UAT, deployment and launch	4–7 days

*Durations are planning estimates and depend on final requirements, content readiness, payment/shipping onboarding and feedback cycles.

19. MVP Launch Scope

- Responsive Nutrexia storefront/PWA.
- Homepage based on approved baseline design.
- Shop and product detail pages.
- Cart and secure checkout.
- Razorpay integration.
- Shipping integration.
- Customer accounts and orders.
- Admin product/order/inventory management.
- Coupons/discounts.
- Reviews.
- Subscription purchase and management.
- SEO foundations and analytics.
- Security, QA and production deployment.

20. Future Expansion (Post-MVP)

- Loyalty/rewards.
- Referral programme.
- Abandoned-cart automation.
- WhatsApp automation.
- Advanced push notifications.
- Gift cards.
- Batch QR verification and public traceability pages.
- Personalised product recommendations.
- Advanced customer analytics.
- Native Android/iOS application if justified by user demand or required device capabilities.

21. Key Business Benefits of the Single-Web-App Approach

- One frontend to develop and maintain.
- Lower initial development and QA effort.
- Same customer experience across desktop and mobile.
- SEO-friendly public storefront.
- Installable app-like mobile experience through PWA.
- One backend, one commerce engine and one database.
- Faster iteration for a 2–3 product launch.
- Native app can be added later without replacing the backend.

22. Client Inputs Required Before Development

- Final logo, brand guidelines, colours and fonts.
- Final product names, variants/flavours, pack sizes and SKUs.
- Final pricing, MRP, discounts and subscription rules.
- Product photographs and lifestyle imagery.
- Final nutrition, ingredient, allergen and regulatory content.
- Lab reports, certificates and documentation approved for publication.
- Shipping/fulfilment details and serviceable regions.
- Return, refund, cancellation, privacy and terms policies.
- Razorpay merchant account and required business verification.
- Shipping-provider account and API access.
- Domain, hosting and production deployment preferences.
- Final approval of claims, testimonials and comparison content before publication.

23. Important Content/Compliance Note

The baseline contains health, nutrition, allergen, lab-testing, climate and product-performance statements. These should be treated as client-supplied content pending final legal/regulatory review and supporting documentation. The development team should implement approved content and provide configurable content management rather than independently validating or inventing claims.

24. Final Recommended Architecture

PostgreSQL → Node.js/NestJS API → Next.js/React responsive PWA

Razorpay and shipping services connect to the backend. The customer uses one frontend across desktop and mobile. The same architecture can later support a native mobile application without changing the core commerce backend.

Prepared as a technical planning document for client discussion. Final specifications may be refined during discovery and UI/UX approval.